Valerie Zhao

(+1) 626-997-6248 | wszhao87@gmail.com | LinkedIn | Portfolio

EDUCATION

University of Washington, Seattle Sep.2024 - Jun.2026(Expected)
Master of Science in Information Management. GPA: 3.9/4.0

China Pharmaceutical University, Nanjing Sep.2020 - Jun.2024
Bachelor of Science in Information Management and Information Systems GPA: 3.7/4.0

SKILLS

Programming Languages: Java, Python, JavaScript, SQL, C++, TypeScript, HTML/CSS
Frameworks & Tools: Spring Boot, React, Vue, Hadoop, Apache Spark, TensorFlow, PyTorch, Git, AuthN/AuthZ
Databases & Caching: MySQL, SQL Server, Redis
Cloud & DevOps: AWS, Azure, Docker, Linux, CI/CD, Nginx
Concepts: Distributed Systems, RESTful APIs, Microservices, System Design, SDLC, Data Structures

WORK EXPERIENCE

IpserLab - Full Stack Intern Jun.2025 - Present

- Engineered modular backend services for a **SaaS** platform using **Java, JavaScript, and MySQL**, with a custom ORM layer for persistence and implemented **RESTful APIs** to enable reliable JSON-based data exchange.
- Built responsive user/admin interfaces with **React.js** and **TailwindCSS**, including dynamic multi-step forms and state management, and integrated **Chart.js** for interactive data visualization.
- Implemented **secure authentication system** with email validation, server-side login, token-based session management, and 2FA support.
- Designed and launched a 6-step dynamic application workflow supporting applicant details, project data, multi-file uploads (PDF, PNG, JPG, DOC), and multi-currency payment integration (USD & Zimbabwean Dollar).
- Delivered usability features including draft-saving & progress tracking, and integrated Email/SMS notifications for automated status updates.

Legend Biotech - IT Department - Full Stack Intern Feb.2023 - Apr.2023

- Developed a cross-platform mobile attendance check-in application using **Java** and **React**, improving employee tracking efficiency.
- Contributed to a tumor treatment tracking system using **Spring Boot** for managing patient data, treatment schedules, and progress tracking, and collaborating on frontend development using **HTML5**, **CSS3**, and **JavaScript**.

PROGRAM EXPERIENCE

Seattle Source Ranker Sep.2025 - Present

- Built a distributed data ingestion system using **Celery** + **Redis** with 16 concurrent workers, enabling large-scale collection of **480K+ GitHub repositories** with an **8.1× throughput improvement** over single-thread execution.
- Engineered a fault-tolerant pipeline with token rotation, automatic retries, rate-limit–aware batching, and resilient message queues, achieving **99.98% reliability** under high-volume, rate-limited workloads.
- Designed a multi-factor ranking algorithm combining log-scaled popularity metrics (stars, forks, watchers) with project age, recent activity, and issue-health scoring for fair, interpretable repo ranking.
- Implemented a scalable data-serving layer generating **9,632 paginated JSON files**, optimized for low-latency static hosting and efficient frontend consumption at large dataset scale.
- Developed an interactive **React** interface featuring lazy loading, debounced search, multi-language filtering, page-jump navigation, and metadata tooltips, enabling real-time exploration of hundreds of thousands of projects.
- Optimized throughput and infrastructure efficiency by reducing API calls to ~1.5 per user, profiling worker concurrency, and minimizing Redis and I/O overhead across the system.

Virtual Character Letter Platform Nov.2024 - Apr.2024

- Developed a hybrid backend using **Spring Boot** for user management and role creation, leveraging **RESTful APIs** for front-end and back-end communication and integrating **LangChain** for **LLM/NLP** support.
- Utilized **Redis** for caching to reduce latency and designed a multi-database architecture, using **MongoDB** for storing unstructured letter content and **MySQL** for managing structured user and role data.
- Implemented **RabbitMQ** to handle asynchronous messaging between Java and Python services, ensuring efficient communication across the system.
- Deployed the application to an **Amazon EC2 instance**, using **Docker** for containerization and **Nginx** for reverse proxy management. Configured **AWS CloudWatch** for monitoring and logging to ensure system stability.